

API Conventions + Resource Inventory

เวอร์ชัน: 0.1 (ร่างแรก) วันที่: 2 สิงหาคม 2569 ขอบเขตเอกสารนี้: กติกากลางของ API ทั้งระบบ + รายการ resource ทั้งหมด ยังไม่รวม: endpoint matrix รายตัว, request/response schema, error catalog เต็ม (อยู่ในตอนที่ 2–5)

อ้างอิง:

- SRS_LTMS_Week05_2.pdf (97 หน้า) — FR 66 ข้อ, BR-01–15, NF, UC-01–05
- LTMS_Database_Design.md — DDL 32 ตาราง (ฉบับ 1 ส.ค. 2569)
- LTMS_State_Machines.html — State Machine 5 entity

ส่วนที่ 1 — API Conventions

1.1 Base URL และ Versioning

https://<api-host>/api/v1

ใส่ v1 ไว้ตั้งแต่แรกเพื่อให้เพิ่ม v2 ได้โดยไม่พัง client เดิม ในเฟสนี้จะมีแค่ v1 เท่านั้น

ทุก endpoint ในเอกสารนี้เขียนแบบย่อโดยละ prefix เช่น POST /teams = POST /api/v1/teams

1.2 การตั้งชื่อ (Naming)

ส่วน	รูปแบบ	ตัวอย่าง
Path segment	kebab-case พหุพจน์	/tournament-applications , /sport-types
JSON field	camelCase	sportTypeId , readinessStatus , createdAt
Query parameter	camelCase	?sportTypeId=3&pageSize=20
Enum value	คงค่าเดิมจาก DB เป๊ะ	"Ready" , "pending_approval" , "onsite"

ข้อควรระวัง: DB ใช้ snake_case แต่ API ใช้ camelCase → ต้องมีชั้นแปลง (mapper) คั่นกลางระหว่าง repository กับ controller ห้ามส่ง row จาก MySQL ออกไปตรงๆ เพราะจะหลุด field ที่ไม่ควรเปิดเผย เช่น passwordHash

ค่า enum ไม่แปลง เพราะเป็นข้อมูล ไม่ใช่ชื่อฟิลด์ — แปลงแล้วจะไปเทียบกับ DB ไม่ตรง

1.3 การยืนยันตัวตน (Authentication)

กลไก: JWT แบบ access token อย่างเดียว (ไม่มี refresh token ในเฟสนี้)

Header ที่ทุกคำขอที่ต้องการสิทธิ์ต้องแนบ:

Authorization: Bearer <token>

Payload ของ token:

{ "sub": "42", "iat": 1754100000, "exp": 1754704800 }

เก็บแค่ sub (user id) เท่านั้น ไม่ฝัง role ลงใน token เพราะ role ของระบบนี้ผูกกับบริบท (BR-02) คนเดียวเป็น Team Leader ในทีมหนึ่ง และ Organizer ในอีกทัวร์นาเมนต์หนึ่งได้พร้อมกัน ฝังลง token ไม่ได้และจะ stale ทันทีที่สิทธิ์เปลี่ยน

อายุ token: 7 วัน ตั้งค่าผ่าน environment variable

```
# .env
JWT_SECRET=<ค่าสุ่มยาว อย่างน้อย 32 ตัวอักษร>
JWT_EXPIRES_IN=7d
```

```
// src/config/auth.ts
export const authConfig = {
  secret: process.env.JWT_SECRET!,
  expiresIn: process.env.JWT_EXPIRES_IN ?? '7d',
};
```

รองรับรูปแบบ 15m / 24h / 7d / 30d — เปลี่ยนค่าแล้ว restart ได้เลย ไม่ต้องแก้โค้ด

Response ตอนล็อกอินสำเร็จ (ถือเป็น object ไว้เพื่อรองรับการเพิ่ม refreshToken ในอนาคตโดยไม่เป็น breaking change)

```
{
  "accessToken": "eyJhbGciOi...",
  "expiresIn": 604800,
  "tokenType": "Bearer",
  "user": { "id": 42, "fullName": "...", "userType": "student" }
}
```

การเช็คสถานะบัญชีทุกคำขอ (บังคับ)

JWT เพิกถอนไม่ได้ — token ที่ออกไปแล้วใช้ได้จนหมดอายุ ดังนั้น middleware ต้อง query users ทุกคำขอเพื่อเช็ค is_suspended ตาม FR-UM-05 ถ้าถูกระงับต้องปฏิเสธทันทีไม่ว่า token จะยังไม่หมดอายุก็ตาม

```
export async function requireAuth(req, res, next) {
  const token = req.headers.authorization?.replace('Bearer ', '');
  if (!token) return res.status(401).json({ error: { code: 'NO_TOKEN' } });

  try {
    const payload = jwt.verify(token, authConfig.secret) as { sub: string };
    const user = await userRepo.findById(Number(payload.sub));

    if (!user || user.is_suspended) {
      return res.status(403).json({ error: { code: 'ACCOUNT_SUSPENDED' } });
    }
    req.user = user;
    next();
  } catch {
    return res.status(401).json({ error: { code: 'TOKEN_EXPIRED' } });
  }
}
```

หมายเหตุสำหรับ defend design: เหตุผลที่ไม่ใช้ refresh token ทั้งที่เป็นมาตรฐานอุตสาหกรรม — ประโยชน์หลักของ refresh token คือจำกัดความเสียหายจาก token ที่เพิกถอนไม่ได้ โดยตั้ง access token ให้สั้นมาก แต่ระบบนี้ query สถานะบัญชีจาก DB ทุกคำขออยู่แล้วตาม FR-UM-05 การระงับบัญชีจึงมีผลทันทีภายในวินาที ไม่ต้องรอ token หมดอายุ ประโยชน์ของ refresh token จึงหายไปเกือบทั้งหมด แลกกับความซับซ้อนที่เพิ่มขึ้นทั้งฝั่ง backend และ frontend

1.4 การตรวจสอบสิทธิ์ (Authorization)

DC-02 บังคับ: ทุกคำขอที่ต้องการสิทธิ์ต้องตรวจฝั่ง server เสมอ — การซ่อนปุ่มบนหน้าเว็บไม่ถือเป็นการควบคุมสิทธิ์

NF-SE-02 บังคับ: ต้องตรวจทั้งระดับระบบและระดับบริบท

สิทธิ์ระดับระบบ (System-Level)

Role	วิธีตรวจ
Guest	ไม่มี JWT — เข้าได้เฉพาะ endpoint ที่ระบุว่าเป็น public
User	JWT ผ่าน + is_suspended = false
Admin	มีแถวใน admin_scopes ที่ตรง scope ของ resource นั้น

สิทธิ์ระดับบริบท (Context-Level) — ตรวจต่อ resource เสมอ

Role	วิธีตรวจ	เก็บจริง/Derived
Team Leader	teams.leader_id = req.user.id	Derived
Organizer	tournaments.requested_by_user_id = req.user.id AND status NOT IN ('pending_approval','rejected')	Derived

Referee	tournament_referees มีแถว user_id = req.user.id และ invitation_status = 'accepted' (+ external_approval_status = 'approved' ถ้า is_external = true)	เก็บจริง
---------	--	----------

รูปแบบการเขียน middleware: แยกเป็น 2 ชั้นเสมอ

```
requireAuth → requireTeamLeader(:id) / requireOrganizer(:id) / requireReferee(:matchId) / requireAdmin(scope)
```

Admin scope มี 2 กลุ่มตามดีไซน์ DB:

- scope_type = 'faculty' — ส่วนกลางคณะ ดูแลทั่วรั้วนาเมนต์ที่ scope_type เป็น department หรือ faculty ของคณะตัวเอง
- scope_type = 'university_wide' — อบก. ดูแล scope_type = 'university'

1.5 HTTP Method และ Status Code

Method	ใช้เมื่อ
GET	อ่านอย่างเดียว ไม่เปลี่ยนสถานะ
POST	สร้าง resource ใหม่ หรือ สั่งให้เกิด state transition (ดู 1.6)
PATCH	แก้ไขบางฟิลด์ของ resource ที่มีอยู่
DELETE	ลบ (ในระบบนี้เกือบทั้งหมดเป็น soft delete)

ไม่ใช่ PUT เพราะไม่มีเคสที่ต้องเขียนทับทั้ง resource

Code	ใช้เมื่อ
200	สำเร็จ มี body
201	สร้างสำเร็จ ตอบ resource ที่สร้าง
204	สำเร็จ ไม่มี body (เช่น unfollow)
400	ข้อมูลที่ส่งมาไม่ถูกต้อง (validation)
401	ไม่ได้ล็อกอิน / token หมดอายุ
403	ล็อกอินแล้วแต่ไม่มีสิทธิ์ / บัญชีถูกระงับ
404	ไม่พบ resource
409	ขัดกับสถานะปัจจุบัน หรือผิด business rule (ดูหมายเหตุ)
422	ผ่าน validation แต่ข้อมูลไม่สมเหตุสมผลเชิงธุรกิจ
429	เรียกถี่เกินกำหนด
500	ข้อผิดพลาดฝั่งเซิร์ฟเวอร์

หมายเหตุเรื่อง 409 vs 422: ระบบนี้มี business rule เยอะมาก (BR-01–15) เพื่อไม่ให้สับสน กำหนดว่า **409** ใช้กับกรณีที่ขัดกับ *สถานะ* ของ resource (เช่น สมัครทัวร์ที่ปิดรับสมัครแล้ว, ลบทีมที่กำลังแข่งอยู่) ส่วน **422** ใช้กับกรณีที่ขัดกับ *กฎเชิงปริมาณหรือคุณสมบัติ* (เช่น BR-05 มีทีม Unofficial ครบ 5 ทีมแล้ว, Hard Filter ไม่ผ่าน)

1.6 Action Endpoint สำหรับ State Transition

หลักการ: ท้ามาใช้ PATCH /resource/:id { "status": "... " } เพื่อเปลี่ยนสถานะ

เหตุผล: state machine ทั้ง 5 entity มี guard คนละแบบและผู้เรียกคนละ role การรวมไว้ใน PATCH เดียวทำให้ต้องเขียน branch ยาวและตรวจสิทธิ์ผิดพลาดง่าย แยกเป็น action endpoint แล้ว 1 endpoint = 1 transition แมปกับ diagram ได้ตรงตัว ทดสอบง่าย และ audit log ชัดเจน

รูปแบบ: POST /<resource>/:id/<action> โดย action เป็นคำกริยา

tournaments			
POST /tournaments/:id/approve	Admin	pending_approval → private	
POST /tournaments/:id/reject	Admin	pending_approval → rejected	(ต้องมี reason)
POST /tournaments/:id/publish	Organizer	private → public	(guard: BR-10)
POST /tournaments/:id/unpublish	Organizer	public → private	
POST /tournaments/:id/open-registration	Organizer	registrationOpen = true	
POST /tournaments/:id/close-registration	Organizer	registrationOpen = false	

auto_deleted (FR-TC-04) และ completed เกิดจาก scheduled job ไม่ใช่ endpoint

teams

DELETE /teams/:id	Team Leader	soft delete, deletedReason='leader_deleted'
-------------------	-------------	---

Forming → Ready เกิดอัตโนมัติเมื่อสมาชิกครบตามกติกา (ตอนรับค่าเชิญ) ไม่มี endpoint แยก no_registration / inactive_6_months เกิดจาก scheduled job (BR-06)

tournament_applications

POST /applications/:id/approve	Organizer	pending → approved (Soft Filter)
POST /applications/:id/reject	Organizer	pending → rejected (ต้องมี reason)
POST /applications/:id/cancel	Team Leader	pending → cancelled ▲ ดู 4.1
POST /applications/:id/withdraw	Team Leader	approved → withdrawn

matches

POST /matches/:id/open-checkin	Organizer/System	scheduled → checkin_open
POST /matches/:id/start	Referee	checkin_open → in_progress

completed เกิดอัตโนมัติเมื่อผลถูก verified / disputed sync มาจาก match_results

match_results

POST /matches/:id/result	ผู้ส่งขึ้นกับ mode (BR-13)	→ submitted
POST /matches/:id/result/verify	อีกฝ่ายที่ไม่ได้ส่ง	submitted → verified
POST /matches/:id/result/dispute	Team Leader / Referee	→ disputed (FR-RS-04)
POST /matches/:id/result/resolve	Organizer	disputed → verified rejected
PATCH /matches/:id/result	Referee / Organizer	verified → verified* (FR-RS-07)

*หมายเหตุสำคัญ — amended ไม่ใช่ค่าใน ENUM match_results.status มีแค่ 4 ค่า: submitted / verified / disputed / rejected การแก้ย้อนหลังตาม FR-RS-07 ไม่เปลี่ยน status แต่เขียนลง 3 คอลัมน์ที่เพิ่มไว้เมื่อ 31 ก.ค. 2569 แทน:

```
UPDATE match_results
SET team_a_score = ?, team_b_score = ?, score_data = ?,
    amended_by_user_id = ?, amend_reason = ?, amended_at = NOW()
WHERE match_id = ?; -- status ยังเป็น 'verified' เหมือนเดิม
```

วิธีตรวจสอบว่าผลนี้เคยถูกแก้: amended_at IS NOT NULL สิ่งที่ API ต้องทำ: ทุก response ที่คืนผลการแข่งขันต้องมี field isAmended (คำนวณจาก amended_at IS NOT NULL), amendedAt และ amendReason เพื่อให้หน้าเว็บแสดงป้าย "ผลนี้มีการแก้ไข" ได้ตาม NF-SE-05 ห้ามรับ status ใน body ของ PATCH นี้ — รับเฉพาะฟิลด์ คะแนน/สถิติ + amendReason (บังคับ) แล้วให้ backend ตั้งค่า amend เอง

team_admin_requests / tournament_amendment_requests ใช้ pattern approve / reject เหมือนกัน

1.7 รูปแบบ Response

สำเร็จ — object เดียว ตอบ object ใดๆ ไม่ต้องห่อ data

```
{ "id": 12, "name": "ทีมวิสาฯ A", "readinessStatus": "Ready" }
```

สำเร็จ — รายการ ห่อด้วย items + pagination

```
{
  "items": [ { "id": 1 }, { "id": 2 } ],
  "pagination": { "page": 1, "pageSize": 20, "totalItems": 137, "totalPages": 7 }
}
```

1.8 รูปแบบ Error

NF-US-03 บังคับ: ข้อความต้องเป็นภาษาไทยที่บอกสาเหตุและวิธีแก้ ไม่ใช่รหัสทางเทคนิค FR-UM-01 บังคับ: ต้องแจ้งข้อผิดพลาดเป็นรายช่อง

```
{
  "error": {
    "code": "VALIDATION_FAILED",
    "message": "ข้อมูลบางช่องไม่ถูกต้อง กรุณาตรวจสอบและกรอกใหม่",
    "fields": {
      "email": "อีเมลนี้ถูกใช้สมัครสมาชิกแล้ว กรุณาใช้อีเมลอื่นหรือเข้าสู่ระบบ",
      "password": "รหัสผ่านต้องมีอย่างน้อย 8 ตัวอักษร และมีตัวเลขอย่างน้อย 1 ตัว"
    }
  }
}
```

- code — คงที่ ภาษาอังกฤษ ใช้ให้ frontend เขียน logic (ห้ามให้ frontend เทียบจาก message)
- message — ภาษาไทย แสดงผลได้ทันที
- fields — มีเฉพาะกรณี validation ไม่มีก็ไม่ต้องใส่คีย์นี้

Error code catalog **เต็มจะทำในตอนที 5** โดยดึงจาก "กรณีผิดพลาด" ที่ระบุไว้แล้วในตาราง FR ของ SRS หัวข้อ 3.1.2–3.1.5

1.9 Validation

DC-03 **บังคับ:** ตรวจทั้งฝั่งหน้าเว็บ (ประสบการณ์ผู้ใช้) และฝั่งเซิร์ฟเวอร์ (ความปลอดภัย) — ใช้ Zod ทั้งคู่

แนวทาง: เขียน Zod schema ไว้ที่ backend เป็นต้นฉบับ แล้วแชร์ type ไปฝั่ง frontend ผ่าน shared package หรือ generate จาก OpenAPI (ตอนที่ 6) เพื่อไม่ให้ 2 ฝั่งเขียนกฎซ้ำแล้วหลุดจากกัน

1.10 Pagination, Filtering, Sorting

Pagination แบบ offset (เพียงพอกับปริมาณตาม SRS หัวข้อ 3.2)

```
?page=1&pageSize=20
```

- page เริ่มที่ 1, ค่า default 1
- pageSize default 20, สูงสุด 100

Sorting

```
?sort=createdAt&order=desc
```

Filtering ใช้ query param ตรงชื่อฟิลด์ เช่น

```
GET /tournaments?sportTypeId=3&status=public&facultyId=5&q=ฟุตบอล
```

q = คำค้นอิสระ (FR-TR-01)

กฎความปลอดภัยของ filter: endpoint สาธารณะต้องบังคับ status = 'public' ที่ชั้น service เสมอ ห้ามให้ client ส่ง status=private มาแล้วเห็นทัวร์นาเมนต์ที่ยังไม่เผยแพร่ (FR-TR-01)

1.11 Idempotency

จำเป็นสำหรับ endpoint ที่ผู้ใช้กดซ้ำได้ง่ายหน้าสนาม โดยเฉพาะช่วงสัญญาณเน็ตไม่ดี (AS-02)

Endpoint	วิธีป้องกัน
POST /matches/:id/checkins	UNIQUE (match_id, user_id) ที่ DB — กดซ้ำตอบ 200 พร้อมข้อมูลเดิม ไม่ใช่ error
POST /matches/:id/resultt	match_results.match_id เป็น UNIQUE อยู่แล้ว — ส่งซ้ำคือ UPDATE แลวเดิม ไม่สร้างแถวใหม่
POST /matches/:id/predictions	UNIQUE (user_id, match_id) อยู่แล้ว
POST /tournaments/:id/applications	UNIQUE (tournament_id, team_id) อยู่แล้ว

หลักการ: การกดซ้ำที่ให้ผลลัพธ์เดิม ต้องตอบสำเร็จ ไม่ใช่ 409 — ป้องกันผู้ใช้สับสนหน้าสนาม

1.12 รูปแบบวันที่และเวลา

- ทุก field วันเวลาใช้ ISO 8601 พร้อม timezone เสมอ: "2026-08-02T14:30:00+07:00"
- field ที่เป็นวันล้วน (เช่น eventStartDate, birthDate) ใช้ "2026-08-02"
- เก็บใน DB เป็น UTC แปลงเป็น Asia/Bangkok ที่ชั้นแสดงผล

1.13 การอัปโหลดไฟล์

ห้ามส่งไฟล์ผ่าน API server ให้ใช้ presigned URL ของ S3 ทั้งหมด เพื่อให้ API server ต้องรับ payload หนักและกิน bandwidth (CO-02 งบประมาณ)

Flow 2 ชั้น:

1. POST /uploads/presign { "purpose": "checkin_photo", "matchId": 88, "contentType": "image/jpeg" }
→ { "uploadUrl": "https://...", "objectKey": "verification/checkins/88/42_photo_online.jpg", "expiresIn": 300 }

2. Client PUT ไฟล์ขึ้น uploadUrl โดยตรง

3. Client ส่ง objectKey กลับมาใน endpoint จริง เช่น POST /matches/88/checkins { "documentS3Key": "..." }

การเข้าถึงไฟล์ ต้องผ่าน presigned URL อายุสั้นเสมอ ห้าม public bucket — โดยเฉพาะ verification/** ที่ NF-SE-03 และ DC-09 (PDPA) บังคับให้เข้าถึงได้เฉพาะกรรมการของแมตช์นั้นและ Admin

1.14 Audit Log

NF-SE-05 บังคับ — การกระทำสำคัญต้องบันทึกว่าใครทำ เมื่อใด ได้แก่: อนุมัติทัวร์นาเมนต์, อนุมัติทีม, ยืนยันผล, แก่สถิติย้อนหลัง, ระบุบัญชี, อนุมัติทีม Official, อนุมัติกรรมการภายนอก

เขียนลง audit_logs ในทรานแซกชันเดียวกับการกระทำนั้น ไม่ใช่ยิงทีหลัง เพื่อไม่ให้เกิดกรณี action สำเร็จแต่ log หาย

1.15 Transaction ที่ห้ามพลาด

จุดที่ NF-RE-01 ระบุว่าต้องเป็น atomic — ถ้าส่วนใดล้มเหลวต้องย้อนทั้งหมด

POST /matches/:id/result/verify เป็น transaction ที่สำคัญที่สุดของระบบ ต้องทำครบในทรานแซกชันเดียว:

1. UPDATE match_results SET status='verified', verified_by_user_id, verified_at
2. UPDATE matches SET status='completed'
3. UPDATE matches SET team_a_id/team_b_id ของ next_match_id (เลื่อนผู้ชนะเข้ารอบถัดไป)
4. UPDATE matches ของ loser_next_match_id ถ้าเป็น Double Elimination
5. UPDATE tournament_standings
6. UPDATE player_profile_stats
7. คำนวณแต้ม Pick'em → INSERT point_transactions + UPDATE users.total_points
8. INSERT notifications ให้ผู้ติดตาม
9. INSERT audit_logs

หลัง commit สำเร็จแล้วค่อย sync MongoDB brackets (metadata การแสดงผลเท่านั้น ไม่ใช่ critical path) ถ้าล้มเหลว retry ได้ไม่กระทบความถูกต้องของระบบ

POST /matches/:id/result/dispute ต้อง UPDATE match_results SET status='disputed' + UPDATE matches SET status='disputed' ในทรานแซกชันเดียวกันเสมอ (match_results เป็น source of truth)

PATCH /matches/:id/result (แก้ไขย้อนหลัง FR-RS-07) ต้องคำนวณผลกระทบใหม่ทั้งหมดในทรานแซกชันเดียว เพราะผลเดิมถูกนำไปใช้คำนวณอย่างอื่นไปแล้ว:

1. UPDATE match_results คำนวณ/สถิติ + amended_by_user_id, amend_reason, amended_at
2. UPDATE player_match_stats ถ้าสถิติรายบุคคลเปลี่ยน
3. คำนวณ tournament_standings ใหม่
4. คำนวณ player_profile_stats ใหม่
5. ถ้าผู้ชนะเปลี่ยน — ต้องแก้ matches.team_a_id/team_b_id ของ next_match_id และ loser_next_match_id ด้วย Δ ถ้ารอบถัดไปแข่งไปแล้วต้องปฏิเสธและให้ Organizer จัดการด้วยมือ (ดู 4.5)
6. คำนวณแต้ม Pick'em ใหม่ → INSERT point_transactions ชดเชย (ห้ามลบรายการเดิม เพื่อรักษา audit trail)
7. INSERT notifications แจ้งทีมที่เกี่ยวข้อง
8. INSERT audit_logs (บังคับตาม NF-SE-05)

1.16 Rate Limiting

จำกัดเฉพาะ endpoint ที่เสี่ยง brute force / spam:

Endpoint	ขีดจำกัด
POST /auth/login	10 ครั้ง / 15 นาที ต่อ IP
POST /auth/register	5 ครั้ง / ชั่วโมง ต่อ IP
POST /auth/forgot-password	3 ครั้ง / ชั่วโมง ต่ออีเมล
POST /tournaments/:id/comments	10 ครั้ง / นาที ต่อผู้ใช้

ส่วนที่ 2 — Resource Inventory

32 ตารางถูกยุบเหลือ 18 resource ตารางที่เป็นความสัมพันธ์ย่อยจะกลายเป็น sub-resource ส่วนตารางที่ระบบคำนวณเองจะไม่มี write endpoint เลย

2.1 ตารางสรุป

#	Resource	Base path	ตาราง MySQL ที่เกี่ยวข้อง	สิทธิ์เขียน	หมายเหตุ
01	Auth	/auth	users , password_reset_tokens	Guest/User	login, register, forgot/reset password
02	Users	/users , /me	users , player_profile_stats	เจ้าของ / Admin	โปรไฟล์สาธารณะ + สถิติเป็น read-only
03	Faculties	/faculties	faculties , departments	Admin	ข้อมูลอ้างอิง เกือบ read-only
04	Sport Types	/sport-types	sport_types	Admin	ข้อมูลอ้างอิง
05	Teams	/teams	teams , team_members , team_invitations , team_admin_requests , official_team_memberships	Team Leader	sub-resource 4 ชุด
06	Tournaments	/tournaments	tournaments , tournament_eligibility_rules	Organizer / Admin	หัวใจของระบบ
07	Tournament Amendments	/tournaments/:id/amendments	tournament_amendment_requests	Organizer → Admin	FR-OM-01
08	Referees	/tournaments/:id/referees	tournament_referees	Organizer / Admin	FR-RM-01–03
09	Applications	/tournaments/:id/applications	tournament_applications	Team Leader / Organizer	FR-TR-03–05
10	Brackets	/tournaments/:id/bracket	MongoDB brackets + matches	Organizer	FR-MM-01
11	Matches	/matches	matches , match_checkins	Organizer / Referee / Player	FR-MM-02–05
12	Match Results	/matches/:id/result	match_results , player_match_stats	Referee / Team Leader	FR-RS-01–07
13	Standings	/tournaments/:id/standings	tournament_standings	ไม่มี write	ระบบคำนวณเอง FR-DL-01–02
14	Announcements	/tournaments/:id/announcements	announcements	Organizer	รวม livestream FR-AN, FR-LS
15	Community	/tournaments/:id/comments ฯลฯ	tournament_feedback	User	แยก 3 endpoint ตาม type
16	Questions	/tournaments/:id/questions	tournament_questions	User / Organizer	FR-OM-08
17	Social	/me/following , /me/notifications	follows , notifications	User	FR-FN-01–03
18	Pick'em	/matches/:id/prediction , /me/rewards	pickem_predictions , rewards , user_rewards , point_transactions	User	FR-PK-01, FR-PP-03
—	Admin Console	/admin/*	admin_scopes , audit_logs	Admin	รวมคิวค่าของทั้งหมด

2.2 ตารางที่ไม่มี endpoint ตรง (จัดการภายใน backend)

ตาราง	เหตุผล
official_team_memberships	เขียนอัตโนมัติตอน Admin อนุมัติคำร้อง Official — บังคับ BR-05 ที่ระดับ DB ผ่าน composite PK
password_reset_tokens	ซ่อนอยู่หลัง /auth/forgot-password และ /auth/reset-password
point_transactions	เขียนอัตโนมัติทุกครั้งที่แต้มเปลี่ยน — เปิดอ่านอย่างเดียวที่ /me/points/transactions
audit_logs	เขียนโดย middleware — Admin อ่านได้ที่ /admin/audit-logs
tournament_standings	ระบบคำนวณเองโดยที่ verified — read-only

tournament_standings	ระบบคำนวณผลแพ้ชนะ — read-only
player_profile_stats	ระบบคำนวณ — read-only ที่ /users/:id/stats

2.3 หมายเหตุการออกแบบที่ต้องตัดสินใจไปแล้ว

/me เป็น alias ของผู้ใช้ปัจจุบัน — /me/notifications ซัดกว่า /users/:id/notifications และกันผลเปิดข้อมูลคนอื่น เพราะไม่มี id ให้เดา

tournament_feedback ถูกแยกเป็น 3 endpoint ทั้งที่เป็นตารางเดียว เพราะ 3 ประเภทมีสิทธิ์ ช่วงเวลาที่เปิด และ validation ต่างกันสิ้นเชิง

POST /tournaments/:id/comments	feedbackType='comment' (รีวิ + คะแนน)
POST /tournaments/:id/organizer-feedback	feedbackType='organizer_feedback' (ถึง Organizer)
POST /tournaments/:id/mvp-votes	feedbackType='mvp_vote' matchId=null
POST /matches/:id/mvp-votes	feedbackType='mvp_vote' matchId=:id

matches ใช้ path 2 แบบ — /tournaments/:id/matches สำหรับดูรายการทั้งทัวร์ (ตารางแข่ง FR-MM-03) และ /matches/:id สำหรับดู/จัดการรายตัว เพราะ match id เป็น global unique อยู่แล้ว ไม่ต้อง nest ให้ path ยาว

Bracket มี endpoint แยกจาก matches ทั้งที่สร้างพร้อมกัน เพราะ POST /tournaments/:id/bracket เป็นการกระทำครั้งเดียวที่สร้าง matches หลายสิบแถว + เอกสาร MongoDB พร้อมกัน ไม่ใช่การสร้าง match ทีละตัว

ส่วนที่ 3 — ลำดับการทำงานต่อ

ตอน	เนื้อหา	สถานะ
0-1	Conventions + Resource Inventory	✓ เอกสารนี้
2	Endpoint matrix ครบทุกตัว (FR / path / role / BR / transition / sprint)	ถัดไป
3	Request/Response schema รายตัว	
4	Error catalog รวม (ภาษาไทย)	
5	แปลงเป็น OpenAPI 3.1 YAML	

ลำดับที่จะไล่ในตอนที่ 2 — ตาม vertical slice ของ Operation Flow ไม่ใช่ไล่ทีละตาราง เพื่อให้ได้เส้นทางที่ใช้งานได้จริงเร็วที่สุด

- 1. Auth + Users (FR-UM) — ทุกอย่างพึ่งพา
- 2. OF-01 ขอจัดตั้งทัวร์ → อนุมัติ → แต่งตั้งกรรมการ → เปิด Public
- 3. OF-02 สร้างทีม → เชิญ → สมัคร → Hard/Soft Filter → สร้าง Bracket
- 4. OF-03 เช็คอิน → บันทึกผล → ยืนยัน → อัปเดต Bracket
- 5. Read-only สำหรับผู้ชม (FR-VW, FR-DL)
- 6. Sprint #1 (Follow, Notification, Community, Pick'em)

ส่วนที่ 4 — ประเด็นค้างที่ต้องตัดสินใจ

4.1 tournament_applications.status ขาดค่า cancelled

DDL ปัจจุบันคือ ENUM('pending','approved','rejected','withdrawn') แต่ State Machine และ FR-TR-04 แยก "ยกเลิกก่อนอนุมัติ" ออกจาก "ถอนตัวหลังอนุมัติ" ซัดเจน

เอกสารนี้ออกแบบ endpoint POST /applications/:id/cancel ไว้แล้วตาม FR ถ้าไม่เพิ่มค่านี้ ต้องยุบ 2 กรณีเป็น withdrawn เหมือนกัน ซึ่งจะแยกไม่ออกว่าทีมยกเลิกเอง ก่อนอนุมัติ หรือถอนตัวหลังได้สิทธิ์แล้ว (กระหนการคืนช่องว่างและการแจ้ง Organizer)

```
ALTER TABLE tournament_applications
  MODIFY COLUMN status
    ENUM('pending','approved','rejected','cancelled','withdrawn')
  NOT NULL DEFAULT 'pending';
```

4.2 UNIQUE key ของ tournament_feedback ยังกันโหวตซ้ำไม่ได้จริง

UNIQUE (tournament_id, match_id, user_id, feedback_type) — MySQL ถือว่าค่า NULL แต่ละแถวต่างกัน ดังนั้นแถวที่ match_id IS NULL (โหวต MVP ระดับทัวร์นาเมนต์, comment, organizer feedback) จะ ไม่ถูกกันซ้ำ โดย unique key นี้เลย

ทางเลือก: ใช้ match_id = 0 แทน NULL สำหรับระดับทัวร์นาเมนต์, หรือเพิ่ม generated column, หรือกันซ้ำที่ application layer แล้วบันทึกไว้เป็น known limitation

ผลต่อ API: endpoint POST /tournaments/:id/mvp-votes ต้อง SELECT เช็คก่อน INSERT เสมอ พึ่ง DB constraint อย่างเดียวไม่ได้

4.3 ⚠ ยังไม่ผ่าน Change Management (SRS หัวข้อ 3.6)

2 เรื่องนี้ต่างจาก SRS ฉบับที่ทีมและอาจารย์เห็นชอบ endpoint ที่เกี่ยวข้องจะติดป้าย ⚠ ไว้ในตอนที่ 2 และตัดออกได้โดยไม่กระทบส่วนอื่น

- 1. Scope ทั้งหมดวิทยาลัย (multi-faculty) — SRS เขียนไว้ระดับคณะ
- 2. `user_type='external'` เป็น Organizer ได้ — SRS ระบุว่า external เป็นได้แค่ Referee

4.4 ค่าที่ยังเป็น placeholder

`sport_types.min_members / max_members` และ `tournaments.dispute_window_hours` ยังไม่ได้ยืนยันค่าจริง กระทบ validation ของ `POST /teams` และช่วงเวลาที่เปิดให้โต้แย้งผล

4.5 ⚠ ขอบเขตของการแก้ผลย้อนหลัง (FR-RS-07) ยังไม่ชัด

SRS ระบุว่าแก้สถิติย้อนหลังได้ แต่ไม่ได้ระบุว่า แก้ได้ถึงเมื่อไหร่

กรณีที่เป็นปัญหา: แมตช์รอบ 8 ทีมถูกแก้ผลจนผู้ชนะเปลี่ยน แต่รอบ 4 ทีมแข่งไปแล้ว → ทีมที่ควรได้เข้ารอบไม่ได้ลงแข่ง แก้ในระบบไม่ได้แล้ว

ข้อเสนอ: ให้ `PATCH /matches/:id/result` ปฏิเสธ (409) ถ้าเข้าเงื่อนไขทั้งสองข้อ — ผู้ชนะเปลี่ยน และ แมตช์ถัดไปมีสถานะเกินกว่า `scheduled` แล้ว ส่วนการแก้ที่ไม่กระทบผู้ชนะ (เช่นแก้สถิติผู้เล่นอย่างเดียว) ทำได้เสมอ

ต้องยืนยันกับทีมว่ารับข้อเสนอนี้ไหม หรือมีนโยบายอื่น